

Rapport TP4 : CouchDB et MapReduce

Abdoulaye Barry

Table des matières

1	Introduction	3
2	Démarrage	3
2.1	Exécution via Docker	3
2.2	Vérification	3
2.3	Interface graphique	3
3	Commandes CRUD via REST	4
3.1	Créer une base de données (PUT)	4
3.2	Lire les informations d'une base (GET)	4
3.3	Insertion de documents	4
3.4	Insertion massive de documents	4
3.5	Récupérer un document par ID (GET)	5
3.6	Supprimer un document (DELETE)	5
4	Fonctionnement de MapReduce	5
4.1	Principe	5
4.2	Exemple : Nombre de films par année	5
4.2.1	Fonction Map	5
4.2.2	Résultats intermédiaires	5
4.2.3	Fonction Reduce	5
4.2.4	Résultats finaux	6
5	Exemple : Nombre de films pour chaque acteur	6
5.1	Fonction Map	6
5.2	Fonction Reduce	6
5.3	Résultats finaux	6

6 Exercice 1 : Représentation et calculs avec MapReduce	7
6.1 Modèle pour représenter la matrice M	7
6.2 Calcul de la norme d'un vecteur	7
6.2.1 Étape Map	7
6.2.2 Étape Reduce	8
6.3 Produit de la matrice M avec un vecteur W	8
6.3.1 Étape Map	8
6.3.2 Étape Reduce	9
7 Conclusion	9

1 Introduction

CouchDB est un système de gestion de bases de données **NoSQL** qui stocke les données sous forme de documents **JSON**. Il offre une interface REST accessible via HTTP, ce qui facilite son intégration dans des applications web.

Le paradigme *MapReduce* est utilisé pour le traitement distribué des données, en deux étapes principales :

- **Map** : Extraction des paires clé-valeur.
- **Reduce** : Agrégation des résultats par clé.

2 Démarrage

2.1 Exécution via Docker

Pour lancer CouchDB dans un conteneur Docker, on utilise la commande suivante :

```
1 docker run -d --name couchdbdemo \  
2   -e COUCHDB_USER=abdoulaye \  
3   -e COUCHDB_PASSWORD=barry \  
4   -p 5984:5984 couchdb
```

Explications :

- **-d** : On exécute en mode détaché.
- **--name couchdbdemo** : On nomme le conteneur.
- **-e** : On définit des variables d'environnement (utilisateur et mot de passe).
- **-p 5984:5984** : On mappe le port **5984** du conteneur sur l'hôte.

Cela télécharge l'image **CouchDB** et lance le service.

Remarque : Si on utilise Docker, il faut penser à **mapper les volumes** pour éviter la perte de données en cas d'arrêt ou de suppression du conteneur.

2.2 Vérification

Pour vérifier que le conteneur est en cours d'exécution, on utilise :

```
1 docker ps
```

2.3 Interface graphique

CouchDB propose une interface web disponible à l'adresse suivante :

`http://localhost:5984/_utils`

Elle permet de gérer les bases et consulter les documents. Une documentation locale intégrée est également disponible.

3 Commandes CRUD via REST

CouchDB utilise les verbes HTTP (**GET**, **PUT**, **POST**, **DELETE**). On peut interagir avec CouchDB à l'aide de l'outil **curl**.

3.1 Créer une base de données (PUT)

```
1 curl -X PUT http://abdoulaye:barry@localhost:5984/films
```

Réponse : `{"ok":true}` si la base est créée.

3.2 Lire les informations d'une base (GET)

```
1 curl -X GET http://abdoulaye:barry@localhost:5984/films
```

On retourne les caractéristiques de la base (nombre de documents, etc.).

3.3 Insertion de documents

Créer un document avec un ID connu (PUT)

```
1 curl -X PUT http://abdoulaye:barry@localhost:5984/films/doc1 \  
2     -d '{"titre":"Inception"}' \  
3     -H "Content-Type: application/json"
```

Créer un document sans ID (POST) CouchDB génère un ID automatiquement :

```
1 curl -X POST http://abdoulaye:barry@localhost:5984/films \  
2     -d '{"titre":"Matrix"}' \  
3     -H "Content-Type: application/json"
```

Remarque : Chaque document possède un champ `_id` unique et un champ `_rev` permettant de suivre les versions des modifications.

3.4 Insertion massive de documents

Pour insérer une collection de documents à partir d'un fichier `films_couchdb.json` :

```
1 curl -X POST http://abdoulaye:barry@localhost:5984/films/_bulk_docs \  
2     -d @films_couchdb.json \  
3     -H "Content-Type: application/json"
```

3.5 Récupérer un document par ID (GET)

```
1 curl -X GET http://abdoulaye:barry@localhost:5984/films/movie:10098
```

3.6 Supprimer un document (DELETE)

```
1 curl -X DELETE http://abdoulaye:barry@localhost:5984/films/doc1
```

4 Fonctionnement de MapReduce

4.1 Principe

- **Map** : Extrait des paires clé-valeur de chaque document.
- **Reduce** : Agrège les résultats par clé.

4.2 Exemple : Nombre de films par année

4.2.1 Fonction Map

Listing 1 – Code de la fonction Map

```
1 function (doc) {  
2     emit(doc.year, doc.title);  
3 }
```

4.2.2 Résultats intermédiaires

```
{ "key": 1977, "value": "La Guerre des étoiles" }  
{ "key": 2003, "value": "Kill Bill : Volume 1" }  
{ "key": 1979, "value": "Apocalypse Now" }  
{ "key": 1992, "value": "Impitoyable" }  
{ "key": 2004, "value": "Eternal Sunshine of the Spotless Mind" }
```

4.2.3 Fonction Reduce

Listing 2 – Code de la fonction Reduce

```
1 function (keys, values) {  
2     return values.length;  
3 }
```

4.2.4 Résultats finaux

```
{ "key": 1977, "value": 1 }
{ "key": 2003, "value": 1 }
{ "key": 1979, "value": 1 }
{ "key": 1992, "value": 1 }
{ "key": 2004, "value": 1 }
```

5 Exemple : Nombre de films pour chaque acteur

5.1 Fonction Map

Listing 3 – Code de la fonction Map pour les acteurs

```
1 function (doc) {
2     for (var i = 0; i < doc.actors.length; i++) {
3         emit({
4             "prenom": doc.actors[i].first_name,
5             "nom": doc.actors[i].last_name
6         }, doc.title);
7     }
8 }
```

5.2 Fonction Reduce

Listing 4 – Code de la fonction Reduce pour les acteurs

```
1 function (keys, values) {
2     return values.length;
3 }
```

5.3 Résultats finaux

```
{ "key": {"prenom": "Mark", "nom": "Hamill"}, "value": 5 }
{ "key": {"prenom": "Harrison", "nom": "Ford"}, "value": 13 }
{ "key": {"prenom": "Carrie", "nom": "Fisher"}, "value": 6 }
{ "key": {"prenom": "Peter", "nom": "Cushing"}, "value": 1 }
{ "key": {"prenom": "Anthony", "nom": "Daniels"}, "value": 4 }
```

6 Exercice 1 : Représentation et calculs avec MapReduce

6.1 Modèle pour représenter la matrice M

La matrice M , de dimension $N \times N$, est utilisée pour représenter les liens entre un ensemble de pages web. Chaque élément M_{ij} de la matrice reflète l'importance du lien entre la page P_i et la page P_j .

Pour représenter cette matrice dans CouchDB, chaque ligne correspond à un document JSON structuré de la manière suivante :

Listing 5 – Structure JSON pour une page web

```
1 {  
2   "page_id": "P1",  
3   "links": [  
4     {"target": "P2", "weight": 0.4},  
5     {"target": "P3", "weight": 0.6},  
6     {"target": "P4", "weight": 0.8}  
7   ]  
8 }
```

Explications :

- `page_id` : Identifiant unique de la page.
- `links` : Liste des cibles avec leurs poids respectifs.

Ce format facilite l'accès aux données et leur traitement avec MapReduce.

6.2 Calcul de la norme d'un vecteur

Chaque ligne i de la matrice peut être interprétée comme un vecteur $V = (v_1, v_2, \dots, v_N)$. La norme d'un tel vecteur est définie par :

$$\|V\| = \sqrt{v_1^2 + v_2^2 + \dots + v_N^2}$$

6.2.1 Étape Map

La fonction Map calcule le carré des poids de chaque lien et émet leur somme pour chaque page :

Listing 6 – Fonction Map pour la norme

```
1 function (doc) {  
2   var sum = 0;  
3   for (var i = 0; i < doc.links.length; i++) {
```

```
4     var weight = doc.links[i].weight;
5     sum += weight * weight;
6 }
7 emit(doc.page_id, sum);
8 }
```

6.2.2 Étape Reduce

La fonction Reduce additionne les résultats intermédiaires et calcule la racine carrée pour obtenir la norme :

Listing 7 – Fonction Reduce pour la norme

```
1 function (keys, values) {
2     var total = 0;
3     for (var i = 0; i < values.length; i++) {
4         total += values[i];
5     }
6     return Math.sqrt(total);
7 }
```

6.3 Produit de la matrice M avec un vecteur W

Le produit d'une matrice M avec un vecteur $W = (w_1, w_2, \dots, w_N)$ donne un vecteur φ défini par :

$$\varphi_i = \sum_{j=1}^N M_{ij} \cdot w_j$$

6.3.1 Étape Map

La fonction Map effectue la multiplication de chaque poids M_{ij} avec l'élément correspondant w_j du vecteur W et émet la somme pour chaque page :

Listing 8 – Fonction Map pour le produit matriciel

```
1 function (doc) {
2     var vectorW = [0.3, 0.5, 0.7, ...]; // Exemple de vecteur W
3     var result = 0;
4     for (var i = 0; i < doc.links.length; i++) {
5         var target = parseInt(doc.links[i].target.replace("P", ""));
6         result += doc.links[i].weight * vectorW[target - 1];
7     }
8     emit(doc.page_id, result);
9 }
```


6.3.2 Étape Reduce

La fonction Reduce additionne les résultats pour chaque ligne de la matrice :

Listing 9 – Fonction Reduce pour le produit matriciel

```
1 function (keys, values) {  
2     var total = 0;  
3     for (var i = 0; i < values.length; i++) {  
4         total += values[i];  
5     }  
6     return total;  
7 }
```

7 Conclusion

Ce rapport détaille un modèle efficace pour représenter des données matricielles dans CouchDB et applique le paradigme MapReduce pour effectuer des calculs avancés comme la norme des vecteurs et le produit matriciel. Cette approche permet de manipuler de grandes quantités de données de manière distribuée et performante.